

```
pid 22139's current affinity list: 1
```

So far, we have seen how an affinity of an already-running process can be set/retrieved. You can also launch a new task with the given affinity. I will show you how you can do so.

```
$ taskset -c 0x01 ./a.out
```

The above command will launch a new task with Affinity Mask 1 and Affinity List 0.

CPU affinity—a developer's perspective

Linux kernel 2.6 supports system calls that your application can exploit to enforce/retrieve the affinity at the process level. These system calls are `sched_setaffinity` and `sched_getaffinity`.

```
#include <sched.h>

int sched_setaffinity(pid_t pid, unsigned int cpusetsize, cpu_set_t *mask);

int sched_getaffinity(pid_t pid, unsigned int cpusetsize, cpu_set_t *mask);
```

While `sched_setaffinity()` sets the CPU affinity, `sched_getaffinity()` retrieves it.

The man page reads, “`sched_setaffinity()` sets the CPU affinity mask of the process whose ID is PID to the value specified by the mask. If PID is zero, then the calling process is used. The argument `cpusetsize` is the length (in bytes) of the data pointed to by the mask. Normally, this argument would be specified as `sizeof(cpu_set_t)`. If the process specified by the PID is not currently running on one of the CPUs specified in the mask, then that process is migrated to one of the CPUs specified in the mask.” On the other hand, “`sched_getaffinity()` writes the affinity mask of the process whose ID is the PID into the `cpu_set_t` structure pointed to by the mask. The `cpusetsize` argument specifies the size (in bytes) of the mask. If PID is zero, then the mask of the calling process is returned.”

On success, `sched_setaffinity()` and `sched_getaffinity()` returns 0, error returns -1.

Let's write a simple line of code to change the CPU affinity of the same program and retrieve its newly assigned affinity.

```
#ifndef _GNU_SOURCE
#define _GNU_SOURCE 1
#endif
#include <stdio.h>
#include <sched.h>

int main()
{
    cpu_set_t mymask;
    unsigned int len = sizeof(mymask);
    unsigned int pid = 0; /* Current Process */

    CPU_ZERO(&mymask);
```

```
/* The 2nd processor in the system */
CPU_SET(1, &mymask);

/* Set affinity mask of the process */
if (sched_setaffinity(pid, len, &mymask) < 0) {
    perror("main: Error in sched_setaffinity()");
    return;
}

CPU_ZERO(&mymask);

/* Get affinity mask of the process */
sched_getaffinity(pid, len, &mymask);

printf("main: My Process Affinity is %p\n", mymask);
return 0;
}
```

The above example binds the current process to the first two processors in the system, and fetches its affinity.

CPU affinity—leftover for developers

Finally, I would like to introduce you to the thread affinity. Initially, it did not seem like it was within the scope of this article, but while researching CPU affinity, I could not find significant discussions on the Internet or elsewhere, on affinity setting/retrieving at the thread level in Linux, apart from a synopsis of APIs in the Linux man page.

So for my own purposes, I did some experiments with thread affinity and believe that it would be worth sharing my experience with you.



Note: I expect readers to be familiar with multi-threading concepts (with *threads*) in Linux.

By default, all threads within a process inherit the same affinity that a process has. We can override this process-level affinity with APIs: `pthread_attr_setaffinity_np()` and `pthread_getattr_np()`.

Let's explore these APIs with our next example.



Note: The *posix* functions with the *_np* suffix are non-standard and not portable.

```
#ifndef _GNU_SOURCE
#define _GNU_SOURCE 1
#endif
#include <pthread.h>
#include <stdio.h>
#include <sched.h>

void *thread_fn(void *arg)
{
    cpu_set_t mymask;
    unsigned int len = sizeof(mymask);
```